Solutions to Advanced Spark Memory & Performance Problems

🔥 Problem 1: Executor OOM During Join

Solution:

1. **Ideal Join Strategy**: Broadcast Hash Join.

2. Because small_df (5MB) is tiny and can fit in executor memory.

3. **Force Broadcast Join**:

```
broadcast_df = broadcast(small_df)
big_df.join(broadcast_df, on="key")
```

Or set:

```
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 10*1024*1024)
```

1. **Default Strategy**:

2. Sort-Merge Join or Shuffle Join, both are memory and disk-heavy, causing OOM.

3. **Ensure Fit**:

4. Use `.count()` and `.storage.memoryUsed()` (via Spark UI or logs).

5. If broadcast fails: increase `spark.sql.broadcastTimeout` or reduce data size.

🦗 Problem 2: Cache Crashes Shuffle

Solution:

1. **What's Happening**:

2. Cached RDDs take up Storage Memory.

3. Shuffle stage needs Execution Memory -> contention leads to eviction or OOM.

4. **spark.memory.fraction**: Default is 0.6

5. You can increase this if you want more storage for cache:

```
spark.conf.set("spark.memory.fraction", 0.8)
```

6. **Use `` with level**:

```
df.persist(StorageLevel.MEMORY_AND_DISK)
```

1. **DISK_ONLY**:

2. Useful when RAM is tight, keeps cache on disk:

```
df.persist(StorageLevel.DISK_ONLY)
```

🐒Problem 3: Skewed Aggregation

Solution:

1. **Issue**: One key dominates data; causes task imbalance, long tail.

2. **Handle Skew**:

3. Use salting technique:

```
df.withColumn("salted_key", concat(col("device_type"), lit("_"),
(rand()*10).cast("int")))
```

Then group by `salted_key` and aggregate, followed by final combine.

4. **Repartition**:

```
df.repartition(200, "device_type")
```

May help balance partitions.

1. **Downside**:

2. Salting makes logic more complex and harder to maintain.

🐐 Problem 4: High Task Deserialization Time

Solution:

1. **Reason**: Large serialized Python functions/UDFs.
2. Too many global variables in scope
3. **Heavy UDFs**:
4. Avoid using lambda or nested function closures
5. Use `@udf` and keep logic outside.
6. **Reduce serialization**:
7. Broadcast large constants
8. Avoid pickling dataframes
9. **Broadcast**:
10. Can avoid large shuffles & reduce deserialization

🏞️Problem 5: Tiny Files

Solution:

1. **Issue**: Small files = FileSystem bottleneck (metadata), poor parallelism
2. **Coalesce Output**:

```
df.coalesce(1).write.parquet(...)
```

Or use:

```
df.repartition(20).write.parquet(...)
```

1. **Downside of small files**:
2. More pressure on HDFS/S3 NameNode
3. Slower reads & job scheduling
4. **repartition vs coalesce**:

5. `coalesce(n)` – narrow, merge to fewer partitions
6. `repartition(n)` – wide, full shuffle

🐴Problem 6: Cache Not Working

Solution:

1. **Why not cached**:
2. Maybe the `.count()` didn't trigger full computation
3. Lineage has changed (you cached a temporary transformation)
4. **Lineage**:
5. If you do:

   ```
   df.filter(...).cache()
   ```

   Then call:

   ```
   df.show()
   ```

   It recomputes from source.
6. **Spark UI**:
7. Look in Storage tab, see # partitions cached
8. **Use **``:

```
df.persist(StorageLevel.MEMORY_AND_DISK)
```

More reliable.

🐖Problem 7: Broadcast Join Fails

Solution:

1. **Reason**:
2. Even though below threshold, broadcast collect failed

3. May be due to serialization or nested structures

4. **Check size**:

```
print(small_df.rdd.map(lambda x: len(pickle.dumps(x))).sum())
```

1. **Fallback**:

```
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)
```

1. **Timeout**:

```
spark.conf.set("spark.sql.broadcastTimeout", 600)
```

🪱Bonus: Memory Leak Investigation

Solution:

1. **Causes**:

2. Unbounded `.collect()`, `.toPandas()`

3. Large accumulators

4. Repeated caching without unpersist

5. **Monitor**:

6. Spark UI: Executor tab → memory usage graph

7. GC logs

8. **Tools**:

9. Spark History Server

10. JVM GC Metrics

11. **Bad Patterns**:

```
df.collect()  # large df to driver
df.toPandas() # for 10M+ rows
```

Use `.limit().collect()` instead

Let me know if you want these wrapped as practice interview questions or coding notebooks with sample data!